DEVELOPER

# PORTFOLIO


HA SEO KYUNG
LOCATION 서울특별시 마포구

MAIL haseokyung6@gmail.com
PHONE 010-8345-1811

## ABOUT ME

# 안녕하세요.
# 데이터와 서비스를 함께 고민하는 개발자 하서경 입니다.

Spring Boot·Node.js 기반 **백엔드** 개발과 AWS·Kubernetes 기반 **클라우드 인프라** 구축 및 CI/CD 운영 경험을 보유한 **신입 엔지니어**입니다.
REST API와 실시간 WebSocket 통신을 구현하고, AWS 환경에서 서비스를 직접 구축·배포하며 백엔드부터 인프라까지 함께 이해하고 문제를 끝까지 해결하는 방식으로 **성장**하고 있습니다.

모르는 것은 먼저 **질문**하고 진행 상황을 **공유**하며, 문제가 발생하면 원인을 끝까지 파고들어 **해결하는 개발자**를 지향합니다.




## 하서경

Backend · Cloud Infrastructure

| | |
|---|---|
| BIRTH | 2002.11.06 |
| GITHUB | https://github.com/sknm1106 |
| BLOG | https://velog.io/@sknm1106/posts |

# PROFILE

## 성신여자대학교 AI융합학부 졸업

- 재학 기간 : 2022.03 ~ 2026.02
- 평균 학점 : **3.91** / 4.50
- 총 이수 학점: 132
- 장학 사항: **성적 우수** 장학금 2회, **해외 인턴십** 장학금

자료구조(A+) 데이터베이스(A+) 컴퓨터비전(A+)
추천시스템(A+) 고급기계학습(A+)

## 기술 스택

| | |
|---|---|
| **Backend** | Java, Spring Boot, Node.js, REST API, WebSocket |
| **Cloud & DevOps** | AWS, Kubernetes, Terraform, Docker, GitHub Actions, ArgoCD |
| **Database & Messaging** | MySQL, MariaDB, Redis, Kafka |
| **AI & ML** | Python, scikit-learn, FastAPI |

## 자격증

| | |
|---|---|
| **2025.12** | SQLD(SQL 개발자) |
| **2025.11** | ADsP(데이터분석준전문가) |
| **2022.03** | 자동차운전면허 2종 보통 |

## 수상 이력

| | |
|---|---|
| **2026.07** | 지능형 애플리케이션 개발자 양성과정 5기 장려상 |
| **2025.12** | 성신여자대학교 IT경진대회 장려상 |

# TIMELINE

## 2022

### AI 전공 입학

AI융합학부에서 소프트웨어와 인공지능 기초 학습을 시작했습니다.

## 2023

### 개발·AI 기초 다지기

Python, C++, 자료구조, 기계학습, 운영체제 등 전공 기반을 쌓았습니다.

## 2024

### 해외 IT 기업 인턴

AutoNSI 에서 ERP·MES·IoT 서비스 구조를 분석하고 소개 자료를 제작했습니다.

## 2025

### 데이터 분석 심화와 웹 개발

LG Aimers 6기 해커톤과 WebSocket 기반 전화 발신 웹서비스 프로젝트를 수행했습니다.

## 2026

### 졸업과 풀스택 역량 확장

성신여자대학교 AI융합학부 졸업 후 프론트엔드, 백엔드, AWS, 보안 학습을 이어가고 있습니다.

# ACTIVITIES

**Training**

## SK쉴더스 루키즈 개발 5기

2026.01 - 2026.07

Spring Boot·React 기반 서비스 개발부터 AWS·Docker·Kubernetes 기반 배포 환경까지 학습하며 백엔드·인프라 역량을 확장했습니다. 최종 프로젝트에서는 클라우드 인프라를 담당해 서비스 연결 과정에서 발생한 Ingress·헬스체크 오류와 Kafka IAM 인증 문제를 원인별로 추적해 해결했습니다. https://github.com/SK-Rookies-AIMS https://github.com/SkRookies-TripProject

**Project**

## 졸업 프로젝트

2025.09 - 2025.12

WebSocket 기반 '콜포비아 사용자를 위한 전화 발신 웹서비스'를 개발하고 백엔드 서버 개발을 담당했습니다. 외부 통화 API의 호출 순서와 CallSid 기반 통화 흐름을 재설계해 통화 상태와 실시간 자막이 사용자별로 동기화되도록 구현했습니다.

https://github.com/Legend-call-bot/front

**Training**

## LG Aimers 6기

2025.01 - 2025.02

머신러닝 교육과 해커톤에 참여해 난임 환자의 임신 성공 여부 예측 모델을 개발하고 데이터 전처리와 Feature Engineering을 담당했습니다. 결측·모순 데이터를 시술 맥락에 맞게 재구성하고 새로운 특징을 설계하며 모델 성능을 개선했습니다.

https://github.com/sknm1106/LG_Aimers_6.git

**Internship**

## AutoNSI 인턴

2024.06 - 2024.08

제조 통합 서비스 ERP·MES·IoT의 구조와 핵심 경쟁력을 분석하고, 서비스 파이프라인을 정리한 PPT와 홍보 영상을 제작했습니다. 제작한 영상은 실제 협력사 미팅에서 활용됐습니다. 서비스를 직접 테스트하며 3건의 개선사항을 담당자에게 전달했습니다.

https://drive.google.com/file/d/18Rr13yoo1du-dt0CtL_s-t8IfXIUvOqq/view?usp=drive_link

## PROJECT

01

### AI 기반 자동차 스마트팩토리 관제 시스템

INFRA/DEVOPS

자동차 제조 현장의 데이터를 실시간으로 수집·분석하여 설비 이상 및 품질 불량을 예측하는 통합 관제 시스템

02

### 콜포비아 사용자를 위한 전화 발신 웹서비스

LEAD/BACKEND

Twilio WebSocket STT LLM TTS를 연결해 사용자의 통화를 실시간 보조

03

### 여행 지출 내역 기록 서비스

BACKEND

여행별로 예산을 설정하고, 지출 내역을 기록·조회·분석할 수 있는 여행 경비 관리 서비스

PROJECT 01 AIMS

# AI 기반 자동차 스마트팩토리 관제 시스템

MSA 기반 제조 이벤트 처리와 관제 대시보드를 운영 가능한 클라우드 환경으로 배포

2026.05.25 ~ 2026.07.29 | 6명
SK쉴더스 루키즈 개발 5기 최종프로젝트

## 프로젝트 배경

- 다수의 알림 발생 시 처리 우선순위 판단 어려움
- 장애 대응 과정에서 시니어의 경험과 노하우에 의존
- 서비스 간 직접 연결 증가로 장애 영향 범위 및 복잡도 증가
- 신규 기능 추가 시 기존 서비스 수정이 필요한 낮은 확장성

→ 기존 제조 관제 환경을 개선하기 위해 **알림 판단 부담을 줄이고 독립적으로 확장 가능한 지능형 관제 시스템**을 기획







## 프로젝트 소개

- 자동차 제조 공정에서 발생하는 실시간 데이터 수집
- AI 분석으로 이상 탐지와 품질 문제를 탐지하는 **스마트팩토리 관제 시스템**
- Kafka 기반 데이터 파이프라인과 RDS Redis ElasticSearch를 활용
- **공정 이상 탐지, 불량 전이 예측, 알림 우선순위화, 실시간 대시보드** 기능을 통해 관제를 지원
- Github: https://github.com/SK-Rookies-AIMS
- 시연영상: https://drive.google.com/file/d/17YbPjQ44zO2ufYhMCduvZGQb2jQmx6Sg/view?usp=sharing

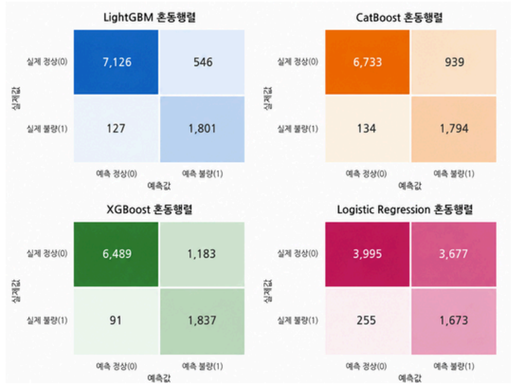
PROJECT 01 AIMS

# AI 기반 자동차 스마트팩토리 관제 시스템

MSA 기반 제조 이벤트 처리와 관제 대시보드를 운영 가능한 클라우드 환경으로 배포

## 주요 기능

1. **현장 대응형 알림 우선순위** - FMEA 개념을 현장 관제 환경에 적용하여 Risk Score, Occurrence, Detection을 반영한 Adaptive eRPN 기반 알림 우선순위 알고리즘을 설계
2. **품질 위험 분석** - ML·SHAP 기반 불량 탐지 및 타 공정 전이 예측
3. **AI 매뉴얼** - RAG·LLM 기반 관제사 숙련도별 점검 순서와 권고 조치 안내
4. **실시간 통합 관제** - WebSocket 공정 흐름, 이상 이벤트, 품질 위험, 조치 상태를 실시간으로 갱신해 상황 판단을 지원




ML 기반 불량 예측




AI 대응 매뉴얼

## 기술 스택

### Frontend·Backend

React, TypeScript, Vite, Node.js
Java, Spring Boot, REST API, WebSocket

### AI / ML

Python, FastAPI, scikit-learn, LightGBM, CatBoost, XGBoost, SHAP, LangChain, OpenAI

### Database & Messaging

MySQL, Kafka, Redis, ElasticSearch

### Cloud Infra & DevOps

AWS(RDS, Redis, MSK, OpenSearch), Terraform, Docker, Kubernetes, GitHub Actions, ArgoCD

### 프로젝트 자료

최종 발표자료 ppt　시스템 아키텍처 정의서　트러블슈팅 개인blog

PROJECT 01 AIMS

# AWS 아키텍처 설계

EKS 서비스와 Kubernetes 배포 구조 설계

## 팀 내 역할

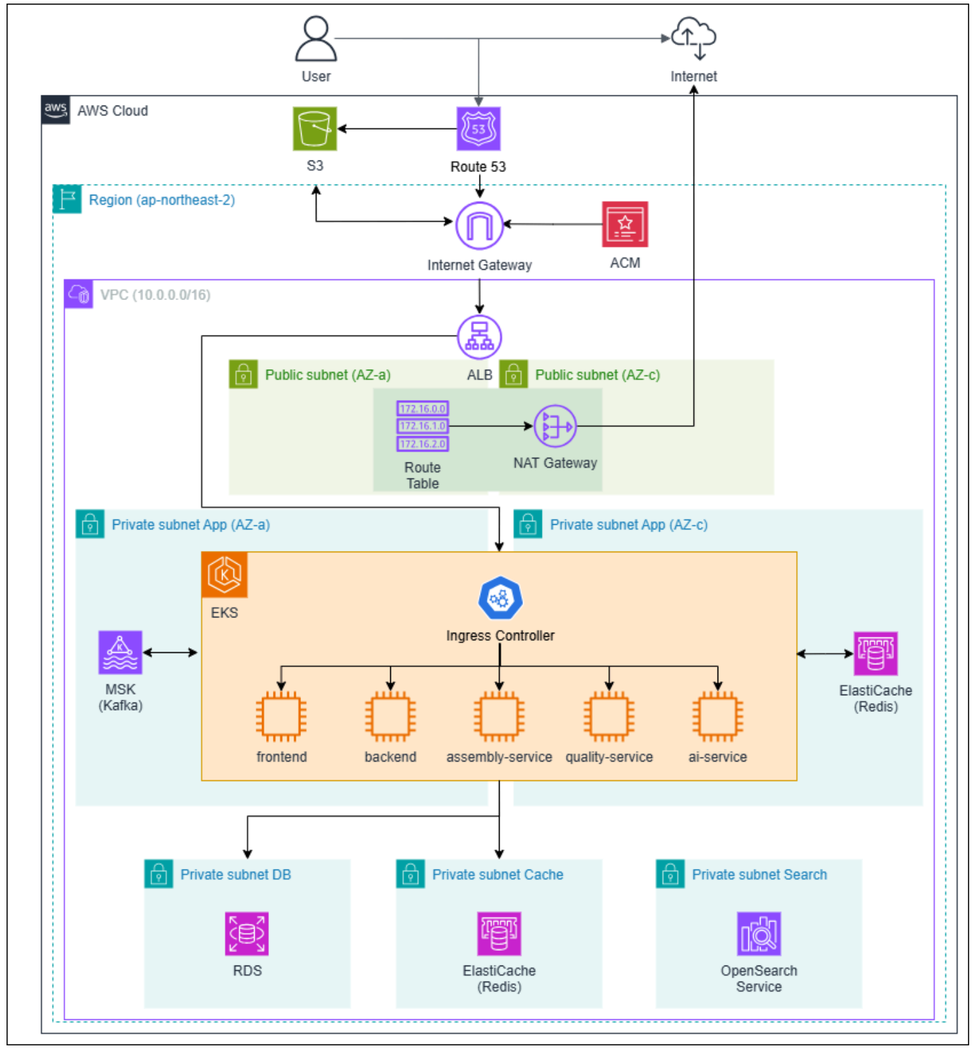
### Cloud Infrastructure · DevOps 전담

- 6명 팀 내 인프라 설계·CI/CD 담당
- VPC·Subnet·IGW·NAT·Route Table 네트워크 구성
- EKS 노드그룹과 Kubernetes 배포 구조 구성
- Route 53·ACM·ALB·Ingress 경로 라우팅 구성
- RDS·Redis·MSK·OpenSearch 접속 구조 설계
- GitHub Actions·ECR·Argo CD 배포 흐름 구성
- SSM·External Secrets·IRSA 기반 시크릿·권한 관리

5개
앱 서비스

HTTPS
오류 해결

IRSA
권한 분리




### 네트워크·트래픽 구성

- Route 53 도메인 연결과 ACM 인증서 적용
- ALB HTTPS 진입점 구성과 Target Group 헬스 체크 관리
- AWS Load Balancer Ingress 기반 경로 라우팅
- 퍼블릭 서브넷과 프라이빗 서브넷 역할 분리
- NAT Gateway 프라이빗 서브넷 아웃바운드 구성

### 애플리케이션·데이터 구성

- EKS 내부 5개 애플리케이션 서비스 배치
- RDS MySQL 영속 데이터 저장
- Redis 기반 캐시와 상태 조회 보조
- MSK Kafka 기반 제조 이벤트 스트리밍
- OpenSearch 기반 분석 결과 조회 구조

PROJECT 01 AIMS

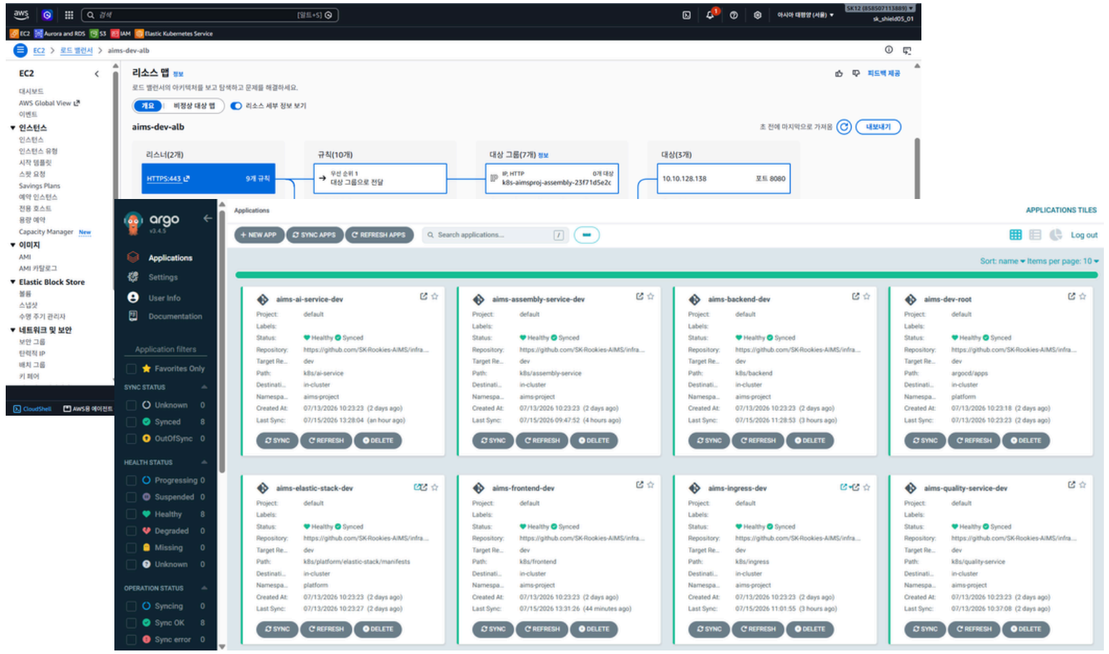
# CI/CD · GitOps · 시크릿 관리 구축

소스코드 변경부터 이미지 빌드, Kubernetes 배포, 민감정보 주입까지 자동화 흐름 구성

Github → Actions Build → ECR → ArgoCD → EKS

## CI/CD 구성 내용

- 프론트엔드·백엔드·조립·품질 · AI 서비스 이미지 빌드 자동화
- GitHub Actions에서 Docker Build·Push 단계 구성
- ECR 이미지 태그 기준 Kubernetes 배포 연계
- Argo CD Application 기반 GitOps 배포 동기화
- Manifest 변경 사항을 Git 기준으로 추적 가능한 구조 구축




SSM Parameter → External Secrets → K8s Secret → ServiceAccount → Pod

## 시크릿 권한 관리

- DB·Redis·Kafka·OpenAI Key 분리 저장
- 소스코드 내 민감정보 제거
- IRSA 기반 AWS 리소스 접근 권한 분리
- ServiceAccount 단위 권한 주입

## 운영 성과

1. 5개 ArgoCD 애플리케이션 정상 동기화
2. 서비스별 CI/CD 파이프라인 구축으로 빌드·배포 과정 자동화
3. Git 기반 Manifest 관리로 배포 이력 및 변경사항 추적 체계화

PROJECT 01 AIMS

# 트러블슈팅 ① 404·502·503 오류 해결

Ingress, Service, Deployment, 헬스체크 설정 불일치를 계층별로 추적

## 문제

- 일부 API 호출 시 404 Not Found 발생
- ALB Target Group의 502 Bad Gateway 발생
- 헬스체크 실패로 인한 503 Service Unavailable 발생
- Pod Running 상태와 외부 접속 실패 상태의 불일치
- 프론트엔드 화면에서 API 데이터 로딩 실패

## 분석 단계

- ALB Target Group 상태와 Health Check Path 확인
- Kubernetes Ingress Path 및 ALB 라우팅 규칙 확인
- Kubernetes Service Port·TargetPort 확인
- Deployment ContainerPort·Application Port 확인
- 애플리케이션 Health API 경로 확인

## 원인

- Ingress 요청 경로와 실제 API 경로 불일치
- Service targetPort와 컨테이너 실행 포트 불일치
- ALB 헬스체크 경로와 애플리케이션 Health API 불일치
- 서비스별 경로 헷갈림
- 배포 단위별 포트 명세 누락

ALB → Ingress → Service → Pod → Health API

## 해결

- 5개 서비스별 **Path·Port·Health Check 매핑표** 작성
- Ingress 경로와 Spring/FastAPI 실제 API 경로 통일
- **Service targetPort와 Deployment containerPort 정합성** 수정
- ALB 헬스체크 경로를 서비스별 **Health API 기준**으로 재설정
- 외부 도메인 기준 경로별 요청 전달 검증 완료

## 운영 성과

1. HTTP 오류 3종 원인 분리와 접속 정상 복구
2. 외부 요청 경로 5개 서비스 기준 정리
3. 배포 후 점검 체크리스트 확보

PROJECT 01 AIMS

# 트러블슈팅 ② Kafka IAM 인증 오류 해결

EKS 재구축 이후 OIDC Provider 변경으로 발생한 IRSA 신뢰 정책 오류 분석

**에러 메세지**

**Not authorized to perform sts:AssumeRoleWithWebIdentity**

## 문제

- EKS 클러스터 재생성 후 AI 서비스 Kafka 연결 실패
- MSK IAM 인증 과정에서 WebIdentity 권한 오류 발생
- Kafka 이벤트 수신 Worker 시작 실패
- 제조 이벤트 분석 중단
- AI 분석 실패

## 분석

- 터미널에서 신규 EKS OIDC Provider 주소 확인
- IAM Role Trust Policy의 Federated Principal 확인
- ServiceAccount annotation의 Role ARN 확인
- Pod 내부 AWS_WEB_IDENTITY_TOKEN_FILE 확인
- Deployment 재시작 후 권한 주입 여부 확인

## 원인과 해결

- 기존 IAM Role이 **이전 OIDC Provider를 참조**
- sub 조건의 namespace·serviceaccount 값 불일치 가능성 점검
- **신규 OIDC Provider 기준 Trust Policy 수정**
- **ServiceAccount와 IAM Role 연결** 정보 재정리
- Deployment rollout restart로 신규 권한 적용

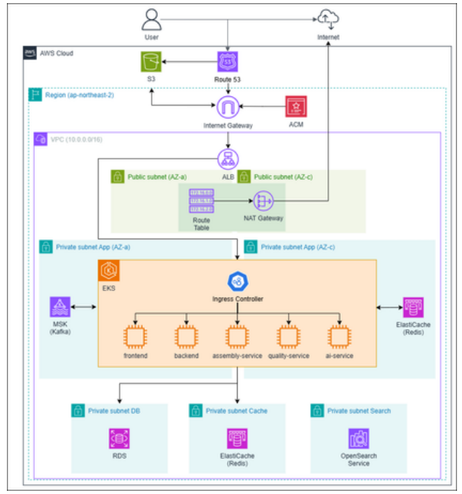
EKS OIDC → IAM Trust Policy → ServiceAccount → IRSA Token → MSK IAM Auth

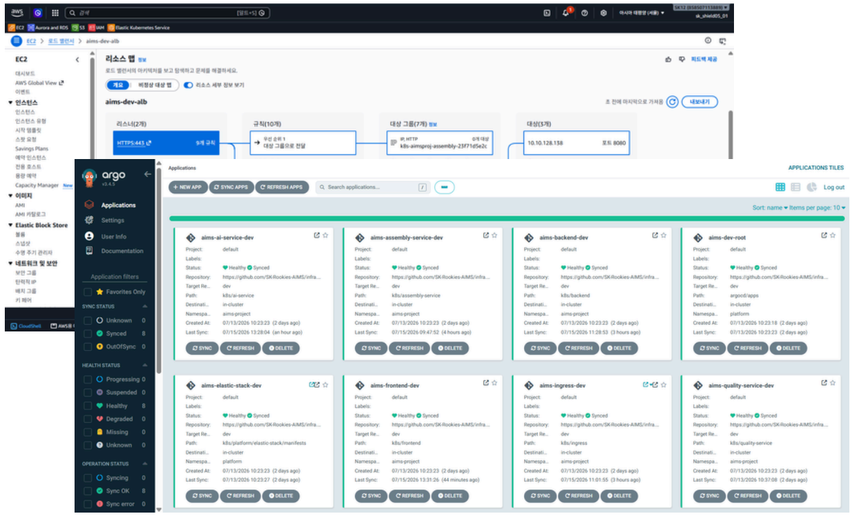
**운영 성과** AI 서비스의 Kafka 연결 오류 해결 및 이벤트 처리 복구

PROJECT 01 AIMS

# 결과 및 성과

인프라 구축, 배포 자동화, 장애 해결 경험 확보






상 장

장려상

교육명 : 지능형 애플리케이션 개발자 양성과정 5기
주제 : AI 기반 자동화 스마트팩토리 관제 시스템
팀명 : Watchy
팀원 : 김건 박지성 이주호 임종찬 최혁민 타시경

위 사람은 SK쉴더스가 주최하는
2026년 지능형 애플리케이션 개발자
양성과정 5기 최종프로젝트 참가하여
우수한 성적으로 입상하였기에
이 상장을 수여합니다.

2026년 7월 30일

SK shieldus



수 료 증

성 명 : 하 시 경
생 년 월 일 : 2002년 11월 6일
교 육 명 : K-Digital 칠단산업 디지털 핵심 실무인력 양성훈련
(디지털 전버빈스) 지능형 애플리케이션 개발자 양성과정)
교 육 기 간 : 2026.01.30 ~ 2026.07.30 120일 (960시간)

위 사람은 인포섹 아카데미에서 실시한 상기
교육 과정을 이수하였으므로 이 증서를 수여
합니다.

2026년 7월 30일

SK shieldus
인포섹 아카데미 총괄기관장

## 운영 성과

1. AWS EKS 기반 MSA 서비스 배포 환경 구축
2. Terraform으로 VPC·EKS·RDS·MSK 등 인프라 구성
3. GitHub Actions·ArgoCD 기반 CI/CD 자동화
4. Kafka IAM·Ingress·ALB 연동 장애 해결
3. 지능형 애플리케이션 개발자 양성과정 5기 수상

## 엔지니어 직무와 연결

**클라우드 인프라 설계 경험**

- Public·Private Subnet 분리 및 EKS 기반 서비스 운영 환경 구성

**장애 원인 분석 경험**

- OIDC·IAM 신뢰 정책과 로그를 추적해 Kafka 인증 오류 해결

**배포 자동화 경험**

- GitHub Actions → ECR → ArgoCD → EKS 배포 흐름 구축

**네트워크·라우팅 운영 경험**

- Route 53·ACM·ALB·Ingress 연계 HTTPS 경로 라우팅 구성

PROJECT 02 CALL BOT

# 콜포비아 사용자를 위한 전화 발신 웹서비스

Twilio·WebSocket·STT·TTS·LLM를 연결한 졸업 프로젝트

2025.09.01 ~ 2025.12.10 | 4명
융합캡스톤디자인 졸업 프로젝트

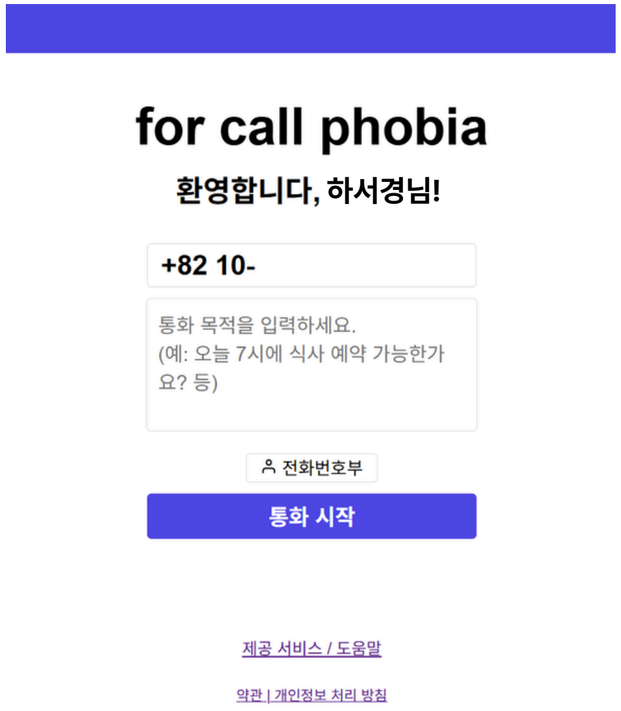
## 프로젝트 소개

- 전화 통화 부담이 큰 사용자의 발신 과정 보조
- 사용자 **의도 입력** 후 서버 기반 **전화 발신**
- 상대방 음성을 실시간 **STT** 텍스트로 변환
- LLM 기반 **추천 답변 3개** 생성
- 선택 문장을 **TTS** 음성으로 상대방에게 재생
- 통화목적·대화내용·요약 결과 저장
- Github: https://github.com/Legend-call-bot/front.git
- 시연영상: https://drive.google.com/file/d/1JFRkyCOSta59SiWZgnYa5holBf-iuIqW/view?usp=sharing

## 팀 내 역할

### Leader · Backend 전담

- Node.js·Express 기반 통화 서버 구현
- Twilio Call API와 Status Callback 연동
- Socket.IO Room과 CallSid 바인딩
- Media Stream·STT·Gemini·TTS 이벤트 흐름 연결
- 통화 기록 저장과 조회 API 구현
- 구버전 SDK 문서 차이 분석과 호출 흐름 재설계




6단계 통화 흐름

4개 외부 서비스

3개 추천 답변

CallSid 이벤트 분리

## 기술 스택

- Node.js·Express·**Socket.IO**
- Twilio Call API·Media Stream
- **Azure** STT·Gemini·ElevenLabs TTS
- PostgreSQL 기반 통화 기록 저장

PROJECT 02 CALL BOT

# 실시간 통화 이벤트 흐름 구현

전화 발신부터 음성 인식, AI 추천, TTS 재생, 요약 저장까지 하나의 흐름으로 연결

**CALL** /calls → **RING** call.accepted → **VOICE** stt.final → **AI** recommendation → **TTS** replySelected/say → **END** summary

## CALL·RING

- 사용자 전화번호·상대방 연락처·통화 목적 검증
- 한국 번호를 E.164 형식으로 정규화
- Twilio Call API 발신 요청
- Status Callback 기반 answered·completed 상태 수신
- 통화 시작 상태를 웹 클라이언트에 실시간 반영

## VOICE·AI

- Twilio Media Stream의 오디오 이벤트 수신
- STT 결과를 실시간 텍스트로 변환
- 통화 히스토리와 최신 발화를 Gemini에 전달
- 상황에 맞는 선택형 추천 답변 3개 생성
- Socket.IO 이벤트로 클라이언트에 추천 답변 전송

## TTS·END

- 사용자가 선택하거나 직접 입력한 문장 수신
- TTS 음성 생성 후 통화 중 상대방에게 재생
- CallSid 기준 통화 이벤트와 스크립트 묶음 처리
- 통화 종료 후 목적·상대방·요약 스크립트 저장
- 기록 대시보드 조회용 데이터 구조 구성

### 실제 전화 발신

Twilio Call API와 status callback으로 answered·completed 상태를 웹에 반영

### 실시간 UI 갱신

Socket.IO room을 callSid에 바인딩해 STT·추천 답변·통화 종료 이벤트를 전달

### 음성->AI->음성

Media Stream 수신 → STT → Gemini 추천 → TTS 재생 파이프라인

### 기록 재사용 기반

Contact upsert와 Call 저장으로 상대방 정보·통화 목적·요약 스크립트 제공

PROJECT 02 CALL BOT

# Backend 구현 범위와 이벤트 분리 구조

실시간 통신에서 사용자가 섞이지 않도록 CallSid 중심으로 이벤트 범위 관리

```
// 추천 버튼 선택
socket.on("replySelected", async ({ text }) => {
    const ctx = getBoundContextOrWarn("replySelected");
    if (!ctx) return;

    const { callSid, userId } = ctx;
```

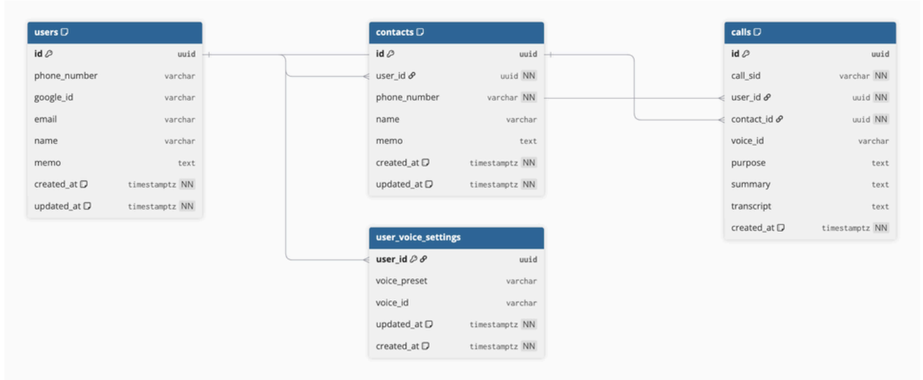
```
// 채팅 입력 say
socket.on("say", async ({ text }) => {
    const ctx = getBoundContextOrWarn("say");
    if (!ctx) {
        socket.emit("say.error", { message: "통화 중이 아닙니다." });
        return;
    }
```

```
await synthesizeToFile(text, filename, { userId, callSid });

const audioUrl = `${PUBLIC_HOST}/audio/${filename}`;
await playToCall(callSid, audioUrl);

pushPlayedText(callSid, text);
pushUserHistory(callSid, text);

socket.emit("say.result", { ok: true });
console.log("🔊 say 재생 성공:", { callSid, filename });
```




## API 구현

- POST /calls 기반 통화 생성 요청 처리
- 전화번호 형식 검증과 E.164 정규화
- 상대방 Contact upsert 처리
- Call 생성과 CallSid 저장
- 통화 종료 후 스크립트·요약 결과 저장
- 기록 조회용 API 응답 구조 구성

## Socket.io 구조

- 클라이언트 접속 시 callSid 기반 room join 처리
- STT 결과 이벤트의 사용자별 분리
- 추천 답변 이벤트의 통화 단위 전송
- TTS 재생 요청 이벤트 say 처리
- 통화 종료 이벤트 completed 전파
- 불필요한 브로드캐스트 방지

## 데이터 흐름

- Contact 정보와 Call 정보 분리 저장
- 통화 목적과 상대방 정보를 함께 보관
- STT 텍스트와 추천 답변의 순서 관리
- 최종 요약 스크립트 저장
- 기록 화면에서 재조회 가능한 구조 구성
- 프론트엔드와 응답 필드 조율

**배운 점** 외부 API · 실시간 이벤트 연동과 로그 기반 장애 분석 경험 확보

PROJECT 02 CALL BOT

# 트러블슈팅 : Twilio SDK와 Media Stream 연결 문제 해결

구버전 예제와 최신 SDK 호출 방식의 차이를 공식 문서 기준으로 재정리

## 문제

- ChatGPT가 구버전 Twilio SDK 기준으로 안내
- 토큰 발급과 Device 생성 순서 혼선
- TwiML App과 Voice Webhook 연결 기준 불명확
- Media Stream 시작 시점 불일치
- 브라우저·Twilio·서버 간 이벤트 순서 추적 어려움

## 분석

- Twilio SDK 공식 문서 기준 최신 호출 흐름 확인
- Access Token 발급 로직과 클라이언트 Device 생성 분리
- TwiML App의 Voice URL과 서버 /voice 라우터 연결 확인
- Media Stream 연결을 /media WebSocket 라우터로 분리
- CallSid 기준 로그를 따라 이벤트 순서 재검증

Token → Device → Call → TwiML → Media Stream → Socket Room

## 해결

- **/token** 라우터로 **클라이언트 토큰 발급 분리**
- **/voice** 라우터에서 **TwiML 응답과 Stream 시작 처리**
- **/media** 라우터에서 **오디오 스트림 수신 처리**
- **CallSid** 기반 **Socket.IO room 연결**
- **발신 · 음성 · 추천·TTS·종료 이벤트 흐름 재정렬**

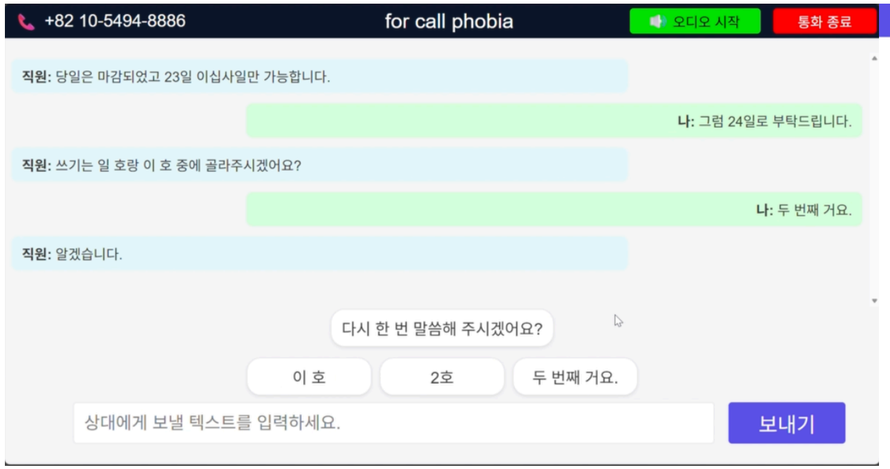
## 운영 성과

1. 실제 전화 발신부터 STT·추천 답변·TTS 재생까지 전체 흐름 연결
2. 호출 순서 문서화와 디버깅 기준 확보
3. 장애 발생 시 SDK·Webhook·서버 라우터를 함께 확인

PROJECT 02 CALL BOT

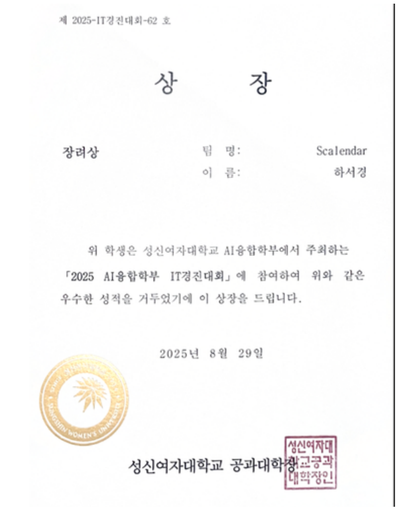
# 결과 및 성과

외부 API 연동과 실시간 이벤트 운영 경험 확보










## 운영 성과

1. 실제 전화 발신부터 종료까지 6단계 이벤트 흐름 구현
2. Twilio·Azure·Gemini·ElevenLabs 4개 외부 서비스 연동
3. 추천 답변 3개 실시간 제공
4. STT·TTS·WebSocket·Socket.IO 실시간 처리 구현
3. 교내 IT 경진대회 수상

## 엔지니어 직무와 연결

**외부 시스템 연동 경험**

- 외부 SaaS API와 내부 Node.js 서버 간 연동 구조 이해

**장애 원인 분석 경험**

- SDK 버전·Webhook·Socket·서버 Route·Callback을 단계적으로 추적

**실시간 서비스 운영 관점**

- 비동기 이벤트의 순서와 상태 동기화 중요성 체득

**로그 기반 디버깅**

- CallSid와 이벤트 로그를 기준으로 장애 흐름 분석

PROJECT 03 COSTRIP

# 여행 지출 내역 기록 서비스

Spring Boot 기반 예산·지출·통계 API 구현으로 애플리케이션과 DB흐름 이해

2026.03.27 ~ 2026.04.08 | 6명
SK쉴더스 루키즈 개발 5기 미니프로젝트

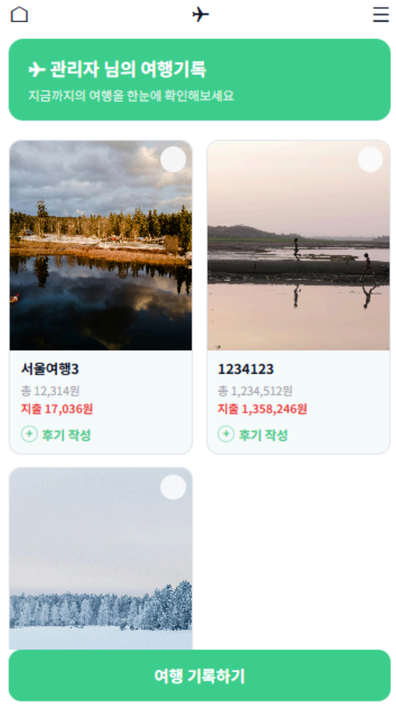
## 프로젝트 소개

- JWT 기반 Stateless 인증·인가
- **여행·예산·지출** 데이터 CRUD
- **카테고리별·일자별 소비 통계** 제공
- 잔여 예산 및 예산 사용률 실시간 계산
- Claude API 기반 영수증 **OCR** 연동
- Flyway 기반 DB 스키마 버전 관리
- Github: https://github.com/SkRookies-TripProject
- 시연영상: https://drive.google.com/file/d/109oAxednPE8bIy-CuRCazRK77-xpdLlF/view?usp=sharing

## 팀 내 역할

### Backend

- Java·Spring Boot 기반 백엔드 기능 구현
- Spring Data JPA와 MariaDB 기반 데이터 처리
- 예산·지출 통계 REST API 구현
- 통계 응답 DTO 설계
- API 연동 과정의 데이터 형식 수정




4종 예산 지표 — 2종 통계 집계 — DTO 응답 구조 — JPA 데이터 처리 — MariaDB 데이터 계층

## 기술 스택

- **Java, Spring Boot,** Spring Data JPA
- Spring Security, JWT
- **MariaDB**, Flyway, REST API
- WebClient, Claude API

PROJECT 03 COSTRIP

# 예산 지출 API 설계

여행 단위 예산 데이터를 조회하고 계산 결과를 프론트엔드에 전달하는 API 구조 구성

Client → Controller → Service → Repository → MariaDB → DTO Response

## 입력 데이터

- 여행 ID 기준 예산 데이터 조회
- 여행 기간과 일자별 지출 데이터 조회
- 지출 카테고리와 금액 데이터 조회
- 사용자별 여행 접근 범위 확인
- 프론트엔드 요청 파라미터 검증

## 계산 로직

- 총예산 조회
- 누적 지출 합계 계산
- 잔여 예산 계산
- 예산 사용률 계산
- 카테고리별 지출 금액 합산
- 일자별 지출 합계 집계

## 응답 설계

- 프론트엔드 차트용 데이터 구조 설계
- 금액·비율·날짜 포맷 정리
- 카테고리별 소비 비중 배열 제공
- 일자별 지출 추이 배열 제공
- 별도 가공 없이 화면 표시 가능한 DTO 구성

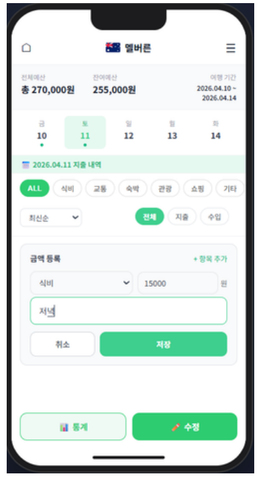
## 대표 API

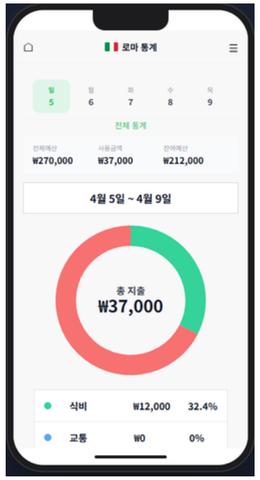
- GET /trips/{tripId}/budget-summary : 예산 요약 조회
- GET /trips/{tripId}/expenses/category : 카테고리별 지출 통계
- GET /trips/{tripId}/expenses/daily : 일자별 지출 추이
- GET /trips/{tripId}/statistics : 통합 통계 조회

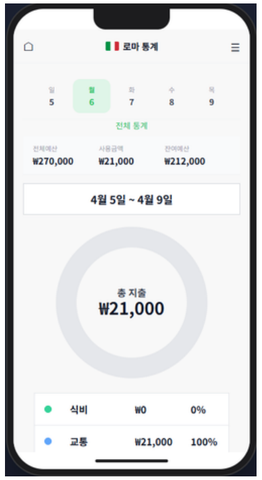
PROJECT 03 COSTRIP

# 통계 집계와 DTO 설계

집계 결과를 화면 단위로 바로 사용할 수 있도록 데이터 형태를 정리







```java
// 1. 카테고리별 합계
List<Object[]> categoryRows = expenseRepository.sumAmountGroupByCategory(tripId);

Map<ExpenseCategory, BigDecimal> categoryAmounts = new LinkedHashMap<>();
BigDecimal grandTotal = BigDecimal.ZERO;

for (Object[] row : categoryRows) {
    ExpenseCategory cat = (ExpenseCategory) row[0];
    BigDecimal amt = (BigDecimal) row[1];

    categoryAmounts.put(cat, amt);
    grandTotal = grandTotal.add(amt);
}
```

```java
// 3. 일자별 합계
List<Object[]> dailyRows = expenseRepository.sumAmountGroupByDate(tripId);
Map<LocalDate, BigDecimal> dailyAmounts = new LinkedHashMap<>();

for (Object[] row : dailyRows) {
    LocalDate date = (LocalDate) row[0];
    BigDecimal amt = (BigDecimal) row[1];
    dailyAmounts.put(date, amt);
}
List<StatisticsResponseDto.DailyExpense> dailyExpenses =
        dailyAmounts.entrySet().stream() Stream<Entry<...>>
                .map( Entry<LocalDate, BigDecimal> e -> StatisticsResponseDto.DailyExpense.builder()
                        .date(e.getKey())
                        .amount(e.getValue())
                        .build()) Stream<DailyExpense>
                .toList();
```

### 예산 API 구현

- 여행별 총예산 조회 API 구현
- 누적 지출 기반 잔여 예산 계산 로직 구현
- 총예산 대비 지출 비율 기반 예산 사용률 계산
- 예산 미설정·지출 없음 등 예외 상황 처리

### 소비 통계 API 구현

- 카테고리별 지출 합계 조회, 소비 비율 계산
- 일자별 지출 합계 조회
- 날짜·카테고리 복합 통계 데이터 구성
- JPQL GROUP BY 기반 집계 쿼리 구현

### API 응답 구조 설계

- 프론트엔드 차트에서 바로 사용할 수 있는 응답 DTO 구성
- 카테고리명·금액·비율 구조 통일
- 날짜별 지출 데이터 정렬
- 통계 API 반환 형식 일관성 확보

### 프론트엔드 연동

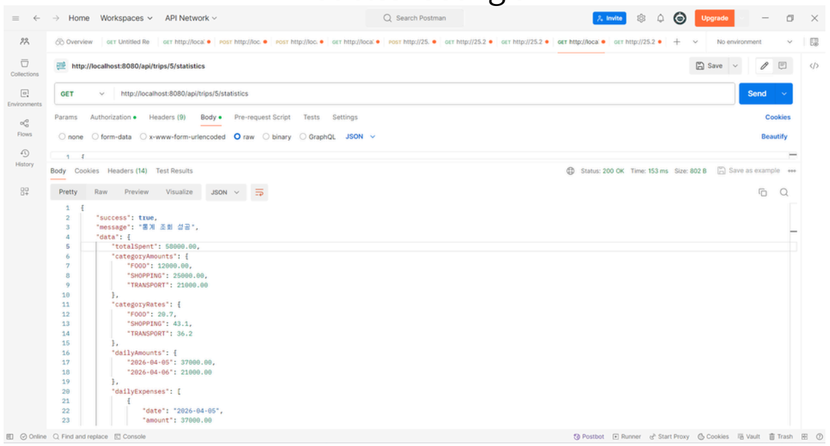
- 백엔드 통계 API 응답 구조에 맞춰 예산·통계 화면 수정
- 프론트엔드 추가 가공을 줄이기 위한 응답 형식 조정
- 예산 현황·카테고리 차트·일자별 추이 화면 연동

PROJECT 03 COSTRIP

# 트러블슈팅 : 통계 집계 구조 개선

집계 방식 변경과 프론트엔드 통계 데이터 가공 최소화

## Troubleshooting 01




### 문제

카테고리별·일자별 통계를 구현할 때 모든 지출 데이터를 조회한 뒤 Java에서 반복문으로 합계를 계산할 경우 데이터 증가에 따라 불필요한 조회와 연산 발생 가능성

### 해결

- ExpenseRepository에 JPQL 집계 쿼리 작성
- **SUM()과 GROUP BY**를 이용한 DB 레벨 집계 적용
- 카테고리별·날짜별 필요한 결과만 조회
- Service 계층에서는 조회 결과를 비율·DTO 형태로 변환하도록 **역할 분리**

**운영 성과** **전체 데이터 조회 방식 → DB GROUP BY 집계 방식으로 개선**

## Troubleshooting 02

```
// 4. 일자별 카테고리별 합계
List<Object[]> dailyCategoryRows = expenseRepository.sumAmountGroupByDateAndCategory(tripId);

Map<LocalDate, Map<ExpenseCategory, BigDecimal>> dailyCategoryAmounts = new LinkedHashMap<>();

for (Object[] row : dailyCategoryRows) {
    LocalDate date = (LocalDate) row[0];
    ExpenseCategory category = (ExpenseCategory) row[1];
    BigDecimal amount = (BigDecimal) row[2];

    dailyCategoryAmounts
            .computeIfAbsent(date, d -> {
                Map<ExpenseCategory, BigDecimal> categoryMap = new LinkedHashMap<>();
                for (ExpenseCategory c : ExpenseCategory.values()) {
                    categoryMap.put(c, BigDecimal.ZERO);
                }
                return categoryMap;
            })
            .put(category, amount);
}
```

### 문제

백엔드에서 단순 집계 값만 전달할 경우 프론트엔드에서 카테고리별 비율 계산·날짜 정렬·차트 데이터 변환 작업 필요

### 해결

- 프론트엔드 추가 계산을 줄이는 응답 구조 설계
- 카테고리별 금액 + 비율 **응답 DTO 설계**
- 일자별 지출 데이터를 날짜 기준 정렬
- 예산 현황을 **총예산·누적 지출·잔여 예산·사용률 구조**로 통일

**운영 성과** **API 응답과 프론트엔드 데이터 구조 일치 → 통계 API 구조 단순화**

DEVELOPER

# 감사합니다


## HA SEO KYUNG

MAIL haseokyung6@gmail.com

PHONE 010-8345-1811